

Exp No.: 9 Monitoring and Logging Fabric Networks

Aim

To monitor the Hyperledger Fabric network by inspecting Docker containers, viewing peer and orderer logs, checking network status, monitoring resource utilization, and analyzing blockchain transaction logs for troubleshooting and performance analysis.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Theory

Monitoring and logging are essential for maintaining a healthy Hyperledger Fabric network. Administrators use monitoring tools and log files to verify that peer nodes, orderers, Certificate Authorities, and chaincodes are functioning correctly. Logs help identify:

- Network startup failures
- Chaincode deployment issues
- Transaction processing errors
- Peer communication failures
- Ordering service problems
- Certificate authentication errors
- Performance bottlenecks

Docker provides built-in commands for monitoring containers and viewing logs, making it easier to troubleshoot Hyperledger Fabric networks.

Procedure

Step 1: Navigate to the Fabric Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Hyperledger Fabric Network

```
./network.sh up createChannel -ca
```

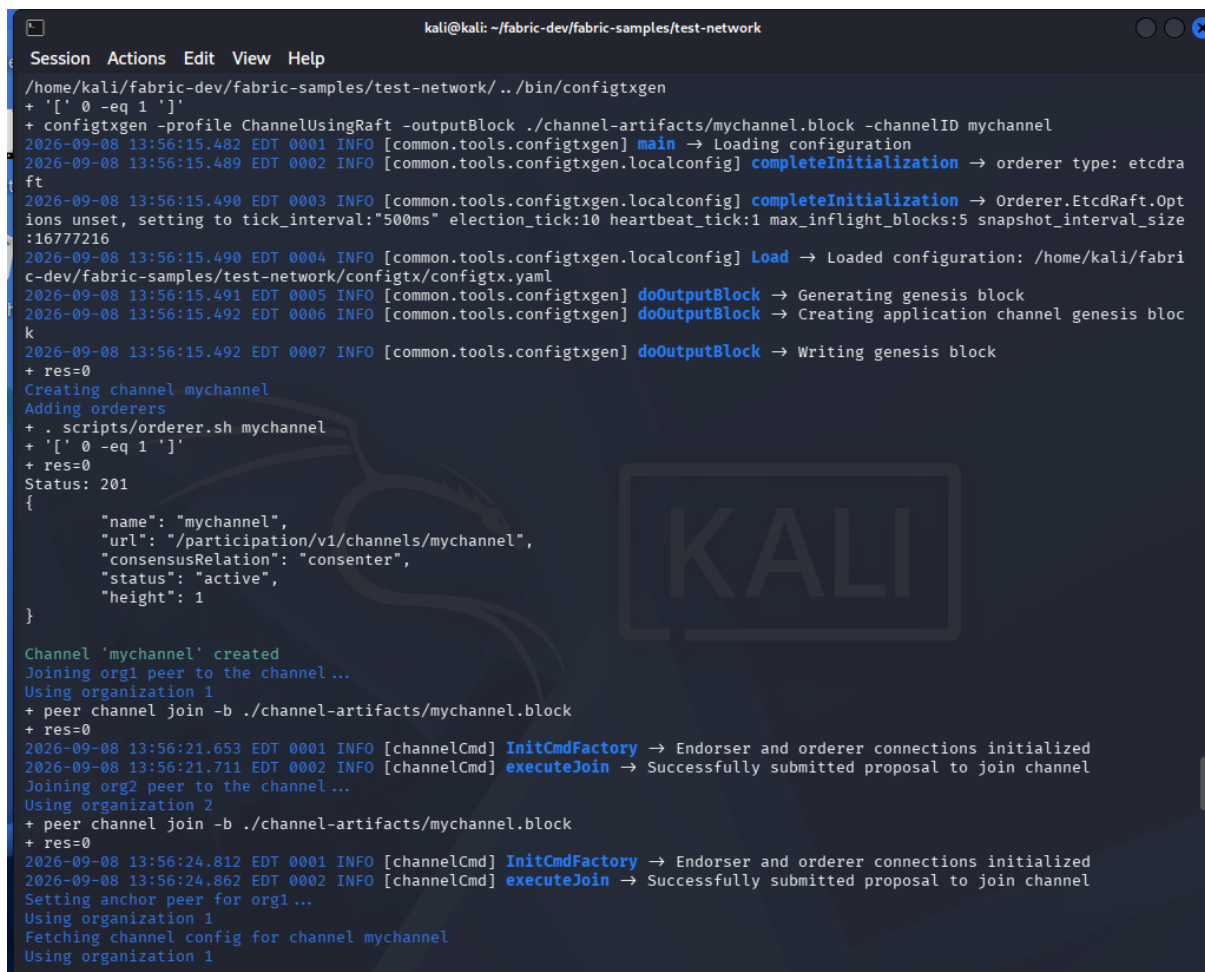
Step 3: Verify Running Containers

Display all active Docker containers.

```
docker ps
```

Step 4: Verify Channel Creation and Peer Joining

Observe the channel genesis block creation, peer organizations joining mychannel, and anchor peer configuration.



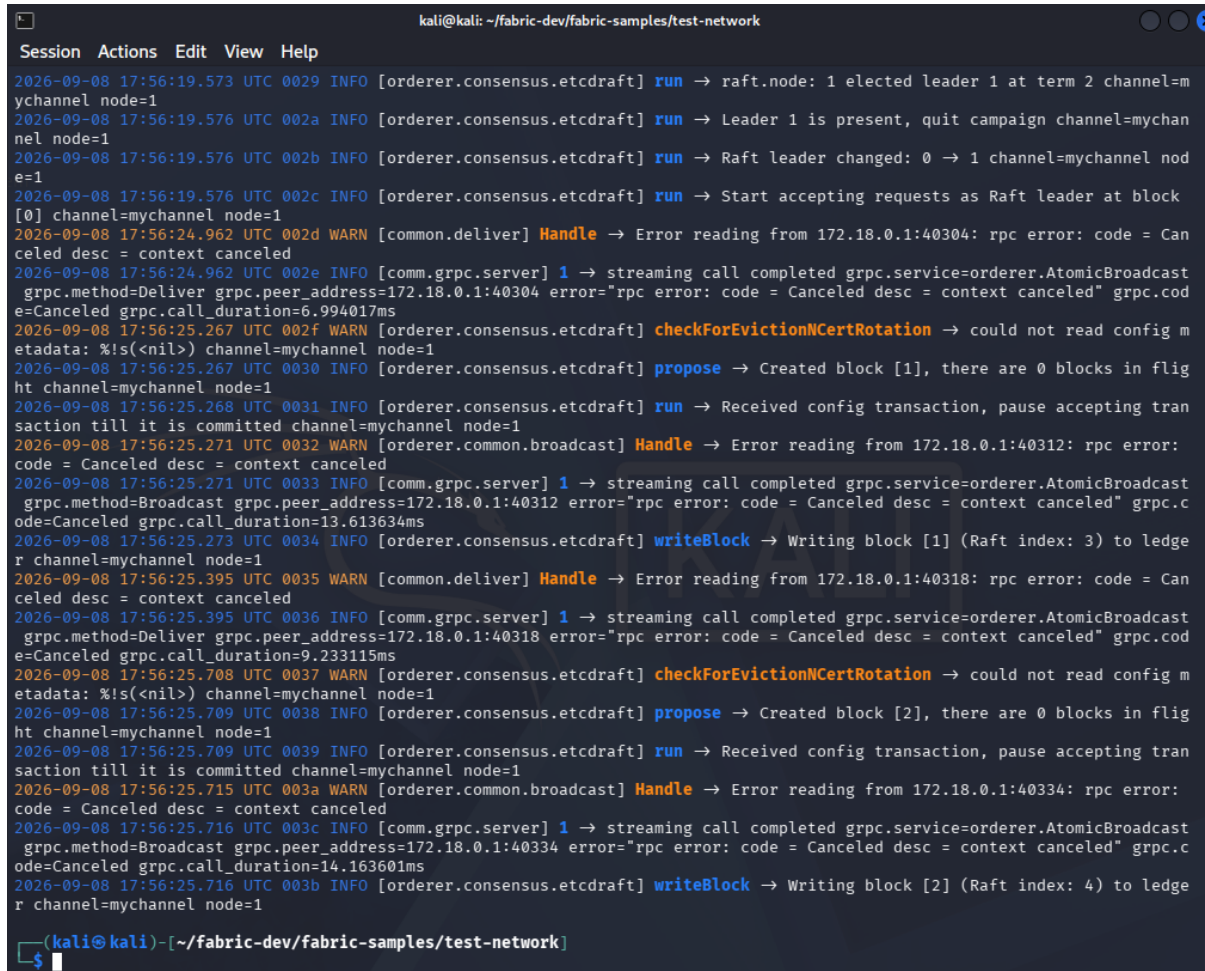
```
kali@kali: ~/fabric-dev/fabric-samples/test-network
Session Actions Edit View Help
/home/kali/fabric-dev/fabric-samples/test-network/ ../bin/configtxgen
+ '[' 0 -eq 1 ']'
+ configtxgen -profile ChannelUsingRaft -outputBlock ./channel-artifacts/mychannel.block -channelID mychannel
2026-09-08 13:56:15.482 EDT 0001 INFO [common.tools.configtxgen] main → Loading configuration
2026-09-08 13:56:15.489 EDT 0002 INFO [common.tools.configtxgen.localconfig] completeInitialization → orderer type: etcdraft
2026-09-08 13:56:15.490 EDT 0003 INFO [common.tools.configtxgen.localconfig] completeInitialization → Orderer.EtcdRaft.Options unset, setting to tick_interval:"500ms" election_tick:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-09-08 13:56:15.490 EDT 0004 INFO [common.tools.configtxgen.localconfig] Load → Loaded configuration: /home/kali/fabric-dev/fabric-samples/test-network/configtx/configtx.yaml
2026-09-08 13:56:15.491 EDT 0005 INFO [common.tools.configtxgen] doOutputBlock → Generating genesis block
2026-09-08 13:56:15.492 EDT 0006 INFO [common.tools.configtxgen] doOutputBlock → Creating application channel genesis block
2026-09-08 13:56:15.492 EDT 0007 INFO [common.tools.configtxgen] doOutputBlock → Writing genesis block
+ res=0
Creating channel mychannel
Adding orderers
+ . scripts/orderer.sh mychannel
+ '[' 0 -eq 1 ']'
+ res=0
Status: 201
{
  "name": "mychannel",
  "url": "/participation/v1/channels/mychannel",
  "consensusRelation": "consenter",
  "status": "active",
  "height": 1
}
Channel 'mychannel' created
Joining org1 peer to the channel...
Using organization 1
+ peer channel join -b ./channel-artifacts/mychannel.block
+ res=0
2026-09-08 13:56:21.653 EDT 0001 INFO [channelCmd] InitCmdFactory → Endorser and orderer connections initialized
2026-09-08 13:56:21.711 EDT 0002 INFO [channelCmd] executeJoin → Successfully submitted proposal to join channel
Joining org2 peer to the channel...
Using organization 2
+ peer channel join -b ./channel-artifacts/mychannel.block
+ res=0
2026-09-08 13:56:24.812 EDT 0001 INFO [channelCmd] InitCmdFactory → Endorser and orderer connections initialized
2026-09-08 13:56:24.862 EDT 0002 INFO [channelCmd] executeJoin → Successfully submitted proposal to join channel
Setting anchor peer for org1...
Using organization 1
Fetching channel config for channel mychannel
Using organization 1
```

Figure 1: Channel mychannel created, Org1 and Org2 peers joining, and anchor peer setup

Step 5: View Orderer Logs

Display the orderer logs to observe Raft consensus, leader election, and block writing.

```
docker logs orderer.example.com
```



```
kali@kali: ~/fabric-dev/fabric-samples/test-network
Session Actions Edit View Help
2026-09-08 17:56:19.573 UTC 0029 INFO [orderer.consensus.etcdraft] run → raft.node: 1 elected leader 1 at term 2 channel=mychannel node=1
2026-09-08 17:56:19.576 UTC 002a INFO [orderer.consensus.etcdraft] run → Leader 1 is present, quit campaign channel=mychannel node=1
2026-09-08 17:56:19.576 UTC 002b INFO [orderer.consensus.etcdraft] run → Raft leader changed: 0 → 1 channel=mychannel node=1
2026-09-08 17:56:19.576 UTC 002c INFO [orderer.consensus.etcdraft] run → Start accepting requests as Raft leader at block [0] channel=mychannel node=1
2026-09-08 17:56:24.962 UTC 002d WARN [common.deliver] Handle → Error reading from 172.18.0.1:40304: rpc error: code = Canceled desc = context canceled
2026-09-08 17:56:24.962 UTC 002e INFO [comm.grpc.server] 1 → streaming call completed grpc.service=orderer.AtomicBroadcast grpc.method=Deliver grpc.peer_address=172.18.0.1:40304 error="rpc error: code = Canceled desc = context canceled" grpc.code=Canceled grpc.call_duration=6.994017ms
2026-09-08 17:56:25.267 UTC 002f WARN [orderer.consensus.etcdraft] checkForEvictionNCertRotation → could not read config metadata: %s(<nil>) channel=mychannel node=1
2026-09-08 17:56:25.267 UTC 0030 INFO [orderer.consensus.etcdraft] propose → Created block [1], there are 0 blocks in flight channel=mychannel node=1
2026-09-08 17:56:25.268 UTC 0031 INFO [orderer.consensus.etcdraft] run → Received config transaction, pause accepting transaction till it is committed channel=mychannel node=1
2026-09-08 17:56:25.271 UTC 0032 WARN [orderer.common.broadcast] Handle → Error reading from 172.18.0.1:40312: rpc error: code = Canceled desc = context canceled
2026-09-08 17:56:25.271 UTC 0033 INFO [comm.grpc.server] 1 → streaming call completed grpc.service=orderer.AtomicBroadcast grpc.method=Broadcast grpc.peer_address=172.18.0.1:40312 error="rpc error: code = Canceled desc = context canceled" grpc.code=Canceled grpc.call_duration=13.613634ms
2026-09-08 17:56:25.273 UTC 0034 INFO [orderer.consensus.etcdraft] writeBlock → Writing block [1] (Raft index: 3) to ledger channel=mychannel node=1
2026-09-08 17:56:25.395 UTC 0035 WARN [common.deliver] Handle → Error reading from 172.18.0.1:40318: rpc error: code = Canceled desc = context canceled
2026-09-08 17:56:25.395 UTC 0036 INFO [comm.grpc.server] 1 → streaming call completed grpc.service=orderer.AtomicBroadcast grpc.method=Deliver grpc.peer_address=172.18.0.1:40318 error="rpc error: code = Canceled desc = context canceled" grpc.code=Canceled grpc.call_duration=9.233115ms
2026-09-08 17:56:25.708 UTC 0037 WARN [orderer.consensus.etcdraft] checkForEvictionNCertRotation → could not read config metadata: %s(<nil>) channel=mychannel node=1
2026-09-08 17:56:25.709 UTC 0038 INFO [orderer.consensus.etcdraft] propose → Created block [2], there are 0 blocks in flight channel=mychannel node=1
2026-09-08 17:56:25.709 UTC 0039 INFO [orderer.consensus.etcdraft] run → Received config transaction, pause accepting transaction till it is committed channel=mychannel node=1
2026-09-08 17:56:25.715 UTC 003a WARN [orderer.common.broadcast] Handle → Error reading from 172.18.0.1:40334: rpc error: code = Canceled desc = context canceled
2026-09-08 17:56:25.716 UTC 003c INFO [comm.grpc.server] 1 → streaming call completed grpc.service=orderer.AtomicBroadcast grpc.method=Broadcast grpc.peer_address=172.18.0.1:40334 error="rpc error: code = Canceled desc = context canceled" grpc.code=Canceled grpc.call_duration=14.163601ms
2026-09-08 17:56:25.716 UTC 003b INFO [orderer.consensus.etcdraft] writeBlock → Writing block [2] (Raft index: 4) to ledger channel=mychannel node=1
(kali@kali) - [~/fabric-dev/fabric-samples/test-network]
$
```

Figure 2: Orderer logs — Raft leader election, block creation, and transaction processing

Step 6: Display Docker Container Statistics

Monitor CPU and memory usage of Fabric containers.

```
docker stats --no-stream
```

Step 7: View Peer Logs

Display the log of Peer Organization 1.

```
docker logs --tail 30 peer0.org1.example.com
```

```

kali@kali: ~/fabric-dev/fabric-samples/test-network
Session Actions Edit View Help
++ jq '.channel_group.groups.Application.groups.Org2MSP.values += {"AnchorPeers":{"mod_policy": "Admins","value":{"anchor_p
eers": [{"host": "peer0.org2.example.com","port": 9051}],"version": "0"}}' /home/kali/fabric-dev/fabric-samples/test-netwo
rk/channel-artifacts/Org2MSPconfig.json
++ res=0
++ configtxlator proto_encode --input /home/kali/fabric-dev/fabric-samples/test-network/channel-artifacts/Org2MSPconfig.jso
n --type common.Config --output /home/kali/fabric-dev/fabric-samples/test-network/channel-artifacts/original_config.pb
++ configtxlator proto_encode --input /home/kali/fabric-dev/fabric-samples/test-network/channel-artifacts/Org2MSPmodified_c
onfig.json --type common.Config --output /home/kali/fabric-dev/fabric-samples/test-network/channel-artifacts/modified_confi
g.pb
++ configtxlator compute_update --channel_id mychannel --original /home/kali/fabric-dev/fabric-samples/test-network/channel
-artifacts/original_config.pb --updated /home/kali/fabric-dev/fabric-samples/test-network/channel-artifacts/modified_config
.pb --output /home/kali/fabric-dev/fabric-samples/test-network/channel-artifacts/config_update.pb
++ configtxlator proto_decode --input /home/kali/fabric-dev/fabric-samples/test-network/channel-artifacts/config_update.pb
--type common.ConfigUpdate --output /home/kali/fabric-dev/fabric-samples/test-network/channel-artifacts/config_update.json
++ jq .
++ cat /home/kali/fabric-dev/fabric-samples/test-network/channel-artifacts/config_update.json
++ echo '{"payload":{"header":{"channel_header":{"channel_id":"mychannel", "type":2}}, "data":{"config_update":{"channel_
id":"mychannel", "isolated_data":{"groups":{"groups":{"Application":{"groups":{"Org2MSP":{"groups":{"mod_policy":{"
mod_policy":{"Admins":{"mod_policy":{"policy": null, "version": "0"}, "Readers":
MSP":{"groups":{"mod_policy":{"policy": null, "version": "0"}, "Readers":
, "version": "0"}, "Endorsement":{"mod_policy":{"policy": null, "version": "0"}, "Readers":
CONTAINER ID   NAME                      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
38eae9535da    peer0.org2.example.com    --        -- / --              --        --        --        --
6b3dcbbb5ec2   peer0.org1.example.com   --        -- / --              --        --        --        --
7d8ba7b74fa5   orderer.example.com      --        -- / --              --        --        --        --
69774a319919   ca_org1                  --        -- / --              --        --        --        --
dbd6dfa6d558   ca_orderer               --        -- / --              --        --        --        --
fa9d3b4c3b1f   ca_org2                  --        -- / --              --        --        --        --

(kali@kali)~/fabric-dev/fabric-samples/test-network
$
(kali@kali)~/fabric-dev/fabric-samples/test-network
$ docker logs --tail 30 peer0.org1.example.com
2026-09-08 17:56:21.708 UTC 0030 INFO [comm.grpc.server] 1 → unary call completed grpc.service=protos.Endorser grpc.method
=ProcessProposal grpc.peer_address=172.18.0.1:42318 grpc.code=OK grpc.call_duration=53.017178ms
2026-09-08 17:56:21.713 UTC 0031 INFO [peer.blocksprovider] DeliverBlocks → Pulling next blocks from ordering service chan
nel=mychannel orderer-address=orderer.example.com:7050 nextBlock=1
2026-09-08 17:56:25.297 UTC 0032 INFO [gossip.privdata] StoreBlock → Received block [1] from buffer channel=mychannel
2026-09-08 17:56:25.312 UTC 0033 WARN [peer.orderers] Update → Config defines both orderer org specific endpoints and glob
al endpoints, global endpoints will be ignored channel=mychannel
2026-09-08 17:56:25.313 UTC 0034 INFO [gossip.gossip] JoinChan → Joining gossip network of channel mychannel with 2 organi
zations
2026-09-08 17:56:25.313 UTC 0035 INFO [gossip.gossip] learnAnchorPeers → Learning about the configured anchor peers of Org
1MSP for channel mychannel: [{peer0.org1.example.com 7051}]
2026-09-08 17:56:25.313 UTC 0036 INFO [gossip.gossip] learnAnchorPeers → Anchor peer for channel mychannel with same endpo
int, skipping connecting to myself

```

Figure 3: Anchor peer config update, container resource statistics, and peer0.org1 logs

Step 8: View Certificate Authority Logs

```
docker logs ca_org1
```

Step 9: Verify Network Channel

```
peer channel list
```

Step 10: Query Installed Chaincode

```
peer lifecycle chaincode queryinstalled
```

Step 11: View Recent Docker Events

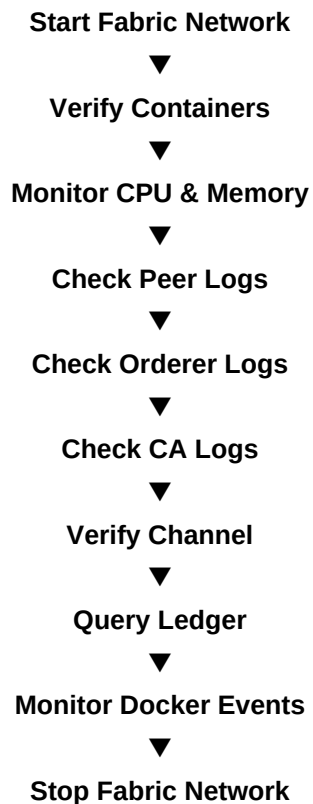
Monitor Docker events generated by the Fabric network.

```
docker events --since 10m
```

Step 12: Stop the Fabric Network

```
./network.sh down
```

Monitoring Workflow



Result

The Hyperledger Fabric network was successfully monitored using Docker utilities and Fabric CLI commands. Peer nodes, ordering service, Certificate Authorities, and blockchain transactions were verified through log analysis and monitoring tools. The experiment demonstrated effective monitoring and troubleshooting techniques for maintaining a reliable Hyperledger Fabric network.